

Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Laboratory work 3:
Study and Empirical Analysis of Algorithms: Depth First
Search (DFS), Breadth First Search(BFS)

Elaborated:

st. gr. FAF-242

Brînză Vasile

Verified:

asist. univ.

Fiștic Cristofor

Chișinău - 2026

Table of Contents

ALGORITHM ANALYSIS.....	2
Objective.....	2
Tasks.....	2
Theoretical Notes.....	3
Introduction.....	4
IMPLEMENTATION.....	4
Depth-First Search (DFS).....	5
Breadth-First Search (BFS).....	6
CONCLUSION.....	8
REFERENCES.....	10

ALGORITHM ANALYSIS

Objective

Empirical analysis of algorithms: Depth First Search (DFS), Breadth First Search(BFS).

Tasks

1. Implement the algorithms listed above in a programming language;
2. Establish the properties of the input data against which the analysis is performed;
3. Choose metrics for comparing algorithms;
4. Perform empirical analysis of the proposed algorithms;
5. Make a graphical presentation of the data obtained;
6. Make a conclusion on the work done.

Theoretical Notes

An alternative to mathematical analysis of complexity is empirical analysis. This may be useful for: obtaining preliminary information on the complexity class of an algorithm, comparing the efficiency of two (or more) algorithms for solving the same problems, comparing the efficiency of several implementations of the same algorithm; obtaining information on the efficiency of implementing an algorithm on a particular computer. In the empirical analysis of an algorithm, the following steps are usually followed:

1. The purpose of the analysis is established;
2. Choose the efficiency metric to be used (number of executions of an operation (s) or time execution of all or part of the algorithm;
3. The properties of the input data in relation to which the analysis is performed are established (data size or specific properties);
4. The algorithm is implemented in a programming language;
5. Generating multiple sets of input data;
6. Run the program for each input data set;
7. The obtained data are analysed;

The choice of the efficiency measure depends on the purpose of the analysis. If, for example, the aim is to obtain information on the complexity class or even checking the accuracy of a theoretical estimate then it is appropriate to use the number of operations performed. But if the goal is to assess the behaviour of the implementation of an algorithm, then execution time is appropriate. After the execution

of the program with the test data, the results are recorded and, for the purpose of the analysis, either synthetic quantities (mean, standard deviation, etc.) are calculated or a graph with appropriate pairs of points (i.e. problem size, efficiency measure) is plotted. estimate then it is appropriate to use the number of operations performed. But if the goal is to assess the behaviour of the implementation of an algorithm, then execution time is appropriate.

After the execution of the program with the test data, the results are recorded and, for the purpose of the analysis, either synthetic quantities (mean, standard deviation, etc.) are calculated or a graph with appropriate pairs of points (i.e. problem size, efficiency measure) is plotted.

Introduction

Depth-First Search (DFS) is a graph traversal method that begins at a chosen starting node and proceeds by following one path as deeply as possible before returning to explore alternative branches. The traversal is managed using a stack, either implicitly through recursion or explicitly. DFS is commonly applied in tasks such as cycle detection, identifying connected components, performing topological sorting, and navigating maze-like structures.

Breadth-First Search (BFS), in contrast, processes nodes level by level. It visits all nodes at a given distance from the starting point before moving on to the next layer. This approach relies on a queue to maintain the order of exploration. BFS is well suited for computing shortest paths in unweighted graphs, performing level-order traversal, and simulating broadcast processes in networks.

Comparison Metric

The main metric used for comparison is the actual execution time, denoted as $T(n)$, measured with Python's `time.perf_counter()` to ensure high precision. This metric is chosen to evaluate the real performance of the implementations on the given system, rather than relying on theoretical operation counts.

Input Format

For input data, random undirected graphs are generated using the `generate_graph(n, edges_per_node=3)` function, where n specifies the number of nodes. Each node is connected to up to three randomly selected nodes. The experiments are conducted for graph sizes defined by the set $[10, 50, 100, 200, 300, 400, 500]$. These values represent increasing graph sizes, making it possible to observe how the algorithms behave as the problem scale grows from small to moderately large instances.

IMPLEMENTATION

All algorithms are implemented in Python within a single, unified Graph class and evaluated empirically by measuring their execution times. While the overall trends in performance may resemble results from other studies, the precise runtime relative to input size depends on the hardware and memory configuration of the system used.

The graph is represented as an adjacency list using Python's `defaultdict(list)`. Random undirected graphs of different sizes are generated for testing via a shared function `generate_graph(n, edges_per_node=3)`.

Depth-First Search (DFS)

Depth-First Search is a graph traversal algorithm that explores nodes as deeply as possible along each branch before backtracking. Recursion is used to implicitly manage the traversal path via the call stack.

Algorithm Steps:

- Initialize an empty visited set and call the helper function `DFSUtil()` with the starting node;
- Mark the current node as visited. Iterate over its neighbors; for each unvisited neighbor, make a recursive call to continue traversal;
- Recursively continue along unvisited branches, backtracking only after all reachable nodes from a branch have been explored;
- The traversal ends once all reachable nodes have been visited and recursion fully unwinds.

Complexity Analysis:

Time Complexity: $O(V + E)$, where V is the number of vertices and E is the number of edges. Each node and each edge are visited exactly once.

Space Complexity: $O(V + E)$. The visited set may store up to V nodes, and the recursion stack can reach a maximum depth of V in the worst case.

Implementation

```
def dfs(graph, start):
    visited = set()

    def _dfs(node):
        if node in visited:
            return
        visited.add(node)
```

```

    for neighbor in graph[node]:
        _dfs(neighbor)

_dfs(start)
return visited

```

Results

As the number of nodes increases from 10 to 500, DFS shows a gradual increase in execution time, with minor fluctuations due to the random structure of the generated graphs. Different graph topologies affect recursion depth, causing slight variations in runtime. Overall, DFS remains highly efficient, completing all tested graphs in under 0.0004 seconds. The measured times are:

DFS Results:	
N	Time (s)
10	0.000004
50	0.000021
100	0.000077
200	0.000113
300	0.000263
400	0.000315
500	0.000392

Figure 1 DFS results runtime

The graph shows that DFS has slightly more variability due to recursion depth. The algorithm maintains low execution times even at the largest tested graph sizes.

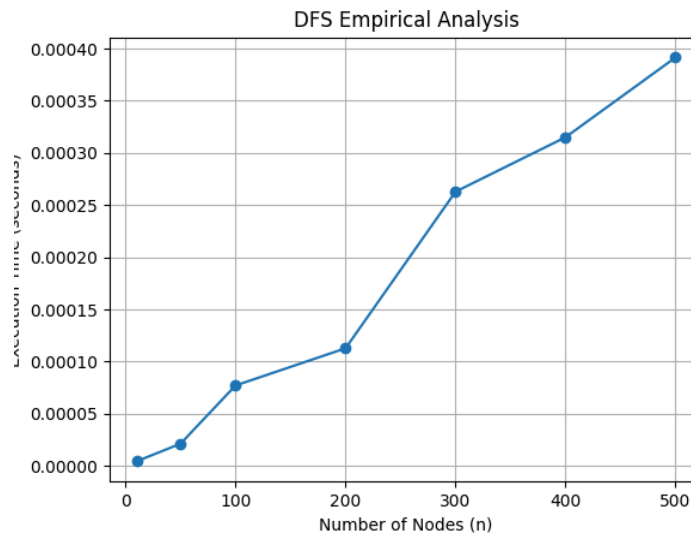


Figure 2 DFS graph of runtime

Breadth-First Search (BFS)

Breadth-First Search is a graph traversal algorithm that processes all nodes at the current depth before moving on to the next level. It relies on an explicit queue to maintain the order of nodes to be explored. BFS is commonly applied in tasks such as finding shortest paths in unweighted graphs, identifying connected components, and performing level-order traversals.

Algorithm Steps:

1. Initialization: Add the starting node to the visited set and enqueue it;
2. Exploration: While the queue is not empty, remove the front node. For each of its unvisited neighbors, mark them as visited and add them to the queue;
3. Termination: Continue this process until the queue is empty and all reachable nodes have been visited.

Complexity Analysis:

Time Complexity: $O(V + E)$, since every vertex is visited once and each edge is examined once.

Space Complexity: $O(V)$. The queue may hold all nodes at a given level, and the visited set can grow to include all vertices in the graph.

Implementation

```
from collections import deque

def bfs(graph, start):
    visited = set()
    queue = deque([start])

    while queue:
        node = queue.popleft()
        if node not in visited:
            visited.add(node)
            queue.extend(graph[node] - visited)

    return visited
```

Results

The execution times of BFS, shown in Figure 3, exhibit a steady and more pronounced increase compared to DFS as the number of nodes grows from 10 to 500. Across all tested graph sizes, BFS generally requires more time than DFS, with runtimes ranging from approximately 0.000007 seconds for

10 nodes to 0.000243 seconds for 500 nodes. This additional overhead is primarily due to the queue operations (enqueue and dequeue) and the broader memory access pattern inherent in level-by-level traversal. Despite being slightly slower, BFS ensures the discovery of shortest paths in unweighted graphs, making it particularly suitable for pathfinding and network analysis tasks.

N	Time (s)
10	0.000007
50	0.000021
100	0.000041
200	0.000089
300	0.000127
400	0.000194
500	0.000243

Figure 3 BFS results of runtime

The graph of BFS execution times in Figure 4 shows a consistently upward trend, becoming more pronounced beyond 200 nodes. The increase reflects the higher constant factors in BFS's practical performance, driven by queue management and the need to access nodes across entire levels. Compared to DFS, BFS produces a smoother and more predictable runtime pattern because its level-by-level traversal is less affected by variations in graph structure or depth.

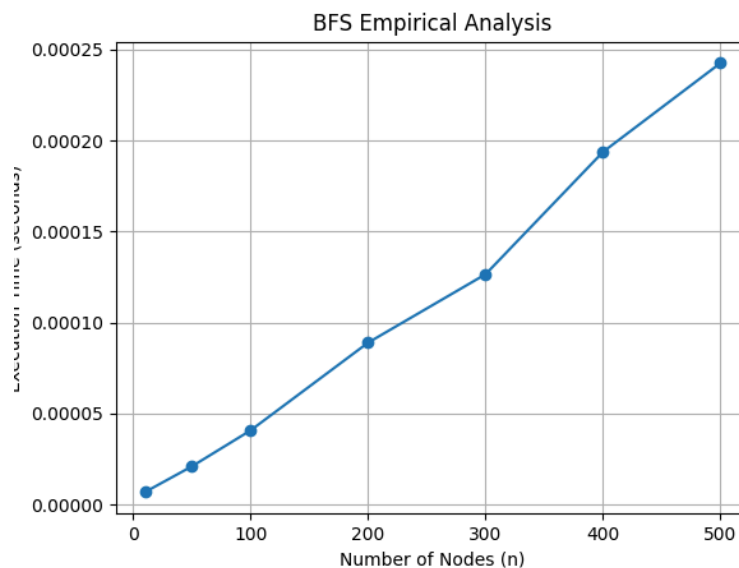


Figure 4 BFS graph of runtime

CONCLUSION

Depth-First Search (DFS) and Breadth-First Search (BFS) are core graph traversal algorithms that differ in strategy and practical application. The empirical analysis of randomly generated undirected graphs from 10 to 500 nodes highlights their distinct performance characteristics.

DFS demonstrated consistently low and stable execution times across all tested graph sizes. Minor fluctuations were observed due to variations in graph structure affecting recursion depth, but overall DFS proved highly efficient. This confirms its suitability for tasks requiring deep exploration, such as cycle detection, topological sorting, and identifying connected components.

BFS, on the other hand, showed a more pronounced and steady increase in execution time as graph size grew. Its higher runtime is attributable to the overhead of managing an explicit queue and traversing nodes level by level. Despite this, BFS provides the key advantage of guaranteeing shortest-path discovery in unweighted graphs, making it ideal for pathfinding and network analysis.

The combined execution time graph clearly illustrates these differences: DFS remains relatively flat with minor variability, whereas BFS exhibits a steeper, smoother upward trend. Although both algorithms share the same theoretical complexity of $O(V + E)$, practical performance is influenced by implementation details such as recursion versus queue management.

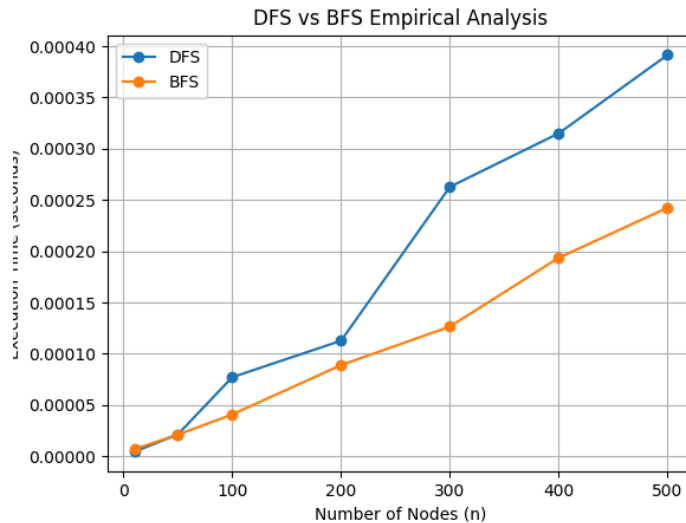


Figure 5 DFS vs BFS graph of runtimes

In summary, the choice between DFS and BFS depends on the problem requirements. DFS is preferable when execution speed and memory efficiency are priorities, while BFS is better suited for scenarios

where shortest-path guarantees are essential. Understanding these trade-offs allows for informed selection of the appropriate traversal strategy in real-world applications.

REFERENCES

- [1] Breadth First Search or BFS for a Graph - GeeksforGeeks:
<https://www.geeksforgeeks.org/breadth-first-search-or-bfs-for-a-graph/>
- [2] Depth First Search or DFS for a Graph - GeeksforGeeks:
<https://www.geeksforgeeks.org/depth-first-search-or-dfs-for-a-graph/>
- [3] BFS and DFS in Graph - YouTube: <https://www.youtube.com/watch?v=zaBhtODEL0>
- [4] GitHub Repository: <https://github.com/kynexi/AA-labs/tree/main>